AIAC_LAB_ASSIGNMENT_17

ASSIGNMENT 17

Name: Chodhitha Maripelli

HT No: 2303A52039

Batch: 31

Task 1: Perform EDA on Penguin dataset (from seaborn)

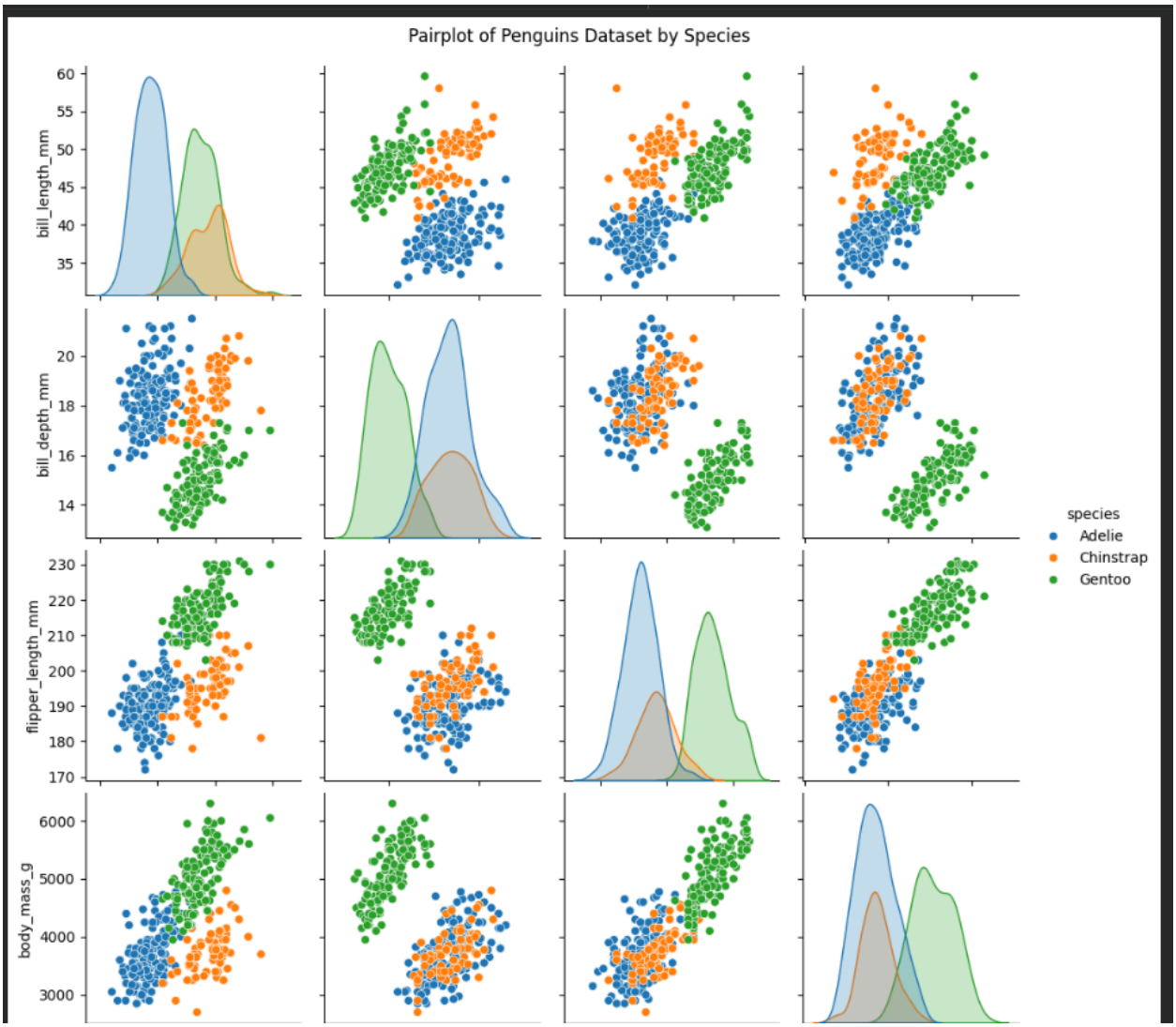
- Load and display entire dataset
- Display country-wise count
- Visualize bar chart (country-wise)
- Visualize bar chart with value counts at the top
- Visualize pairplot and other plots

```
import seaborn as sns
import pandas as pd

# Load the penguins dataset
penguins_df = sns.load_dataset('penguins')
```

```
import seaborn as sns
import matplotlib.pyplot as plt

# Create a pairplot of the penguins dataset, colored by 'species'
sns.pairplot(penguins_df, hue='species')
plt.suptitle('Pairplot of Penguins Dataset by Species', y=1.02) # Add a title above the plot
plt.show()
```



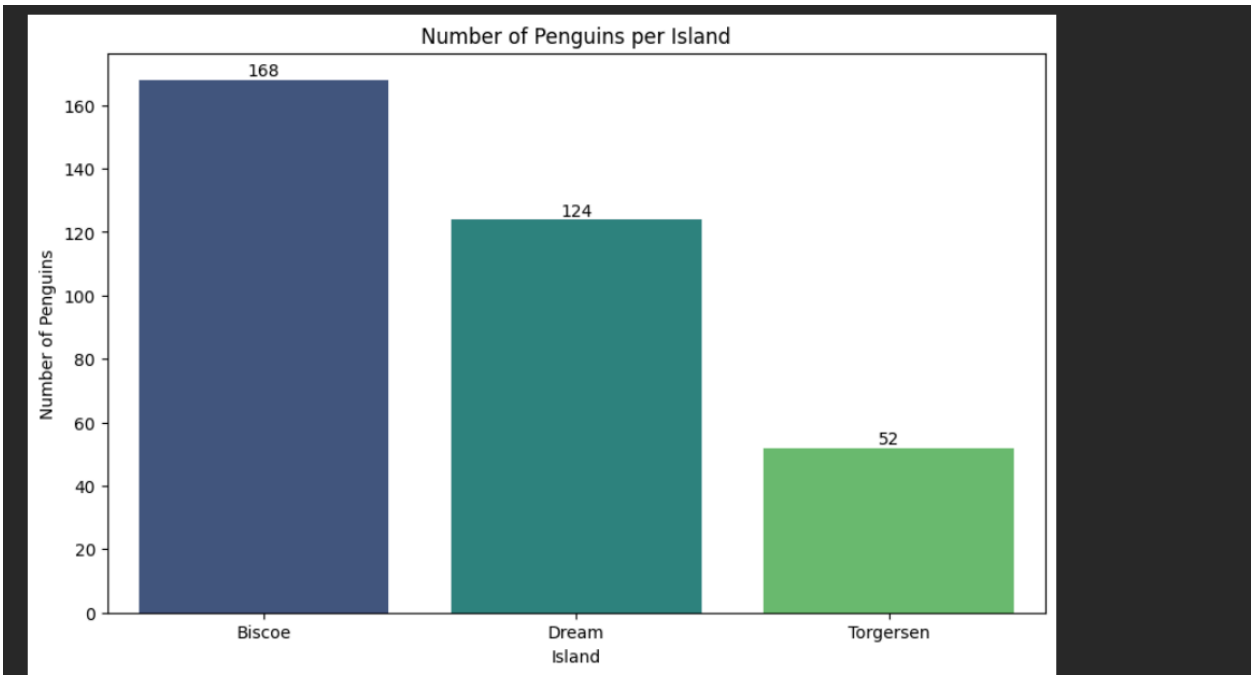

```
import matplotlib.pyplot as plt
import seaborn as sns

# Count the number of penguins per island
island_counts = penguins_df['island'].value_counts().reset_index()
island_counts.columns = ['Island', 'Count']

# Create the bar chart
plt.figure(figsize=(10, 6))
ax = sns.barplot(x='Island', y='Count', hue='Island', data=island_counts, palette='viridis', legend=False)

# Add value counts on top of each bar
for container in ax.containers:
    ax.bar_label(container, fmt='%d', label_type='edge')

plt.title('Number of Penguins per Island')
plt.xlabel('Island')
plt.ylabel('Number of Penguins')
plt.show()
```




```
print(penguins_df.columns)
```

```
Index(['species', 'island', 'bill_length_mm', 'bill_depth_mm',
       'flipper_length_mm', 'body_mass_g', 'sex'],
      dtype='object')
```

```
# Display the first 5 rows of the DataFrame
display(penguins_df.head())
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female

Task 3: Perform the following tasks on the given dataset (train the model)

- Analyze the Planets dataset to understand its structure, features, and missing values
- Perform data cleaning and preprocessing to handle missing or incorrect data

- Train the dataset using different machine learning models to make predictions
- Compare and explain the performance of all trained models with proper reasoning

```python
import pandas as pd
import seaborn as sns
# Load the Planets dataset from seaborn
planets_df = sns.load_dataset('planets')
print("Dataset loaded successfully. Here are the first 5 rows:")
display(planets_df.head())
```

```
Dataset loaded successfully. Here are the first 5 rows:
```

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	269.300	7.10	77.40	2006
1	Radial Velocity	1	874.774	2.21	56.95	2008
2	Radial Velocity	1	763.000	2.60	19.84	2011
3	Radial Velocity	1	326.030	19.40	110.62	2007
4	Radial Velocity	1	516.220	10.50	119.47	2009

Dataset Structure and Features

```python
# Display basic information about the dataset, including data types and non-null values
print("\n--- Dataset Information ---")
planets_df.info()
# Display the shape of the dataset (number of rows, number of columns)
print("\n--- Dataset Shape ---")
print(f"Rows: {planets_df.shape[0]}, Columns: {planets_df.shape[1]}")
# Display column names and their data types
print("\n--- Column Names and Data Types ---")
print(planets_df.dtypes)
```

```
--- Dataset Information ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1035 entries, 0 to 1034
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   method          1035 non-null   object
 1   number          1035 non-null   int64
 2   orbital_period  992 non-null    float64
 3   mass            513 non-null    float64
 4   distance        808 non-null    float64
 5   year            1035 non-null   int64
dtypes: float64(3), int64(2), object(1)
memory usage: 48.6+ KB

--- Dataset Shape ---
Rows: 1035, Columns: 6

--- Column Names and Data Types ---
method             object
number              int64
orbital_period    float64
mass              float64
distance          float64
year                int64
dtype: object
```

Missing Values Analysis

Preparing Data for Model Training

```python
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
import numpy as np
# Define target variable (y) and features (X)
# We'll predict 'mass' as it's a continuous variable suitable for regression
y = planets_df['mass']
X = planets_df.drop('mass', axis=1)
# Identify categorical and numerical features
categorical_features = ['method']
numerical_features = ['number', 'orbital_period', 'distance', 'year']
# Create a column transformer for preprocessing
preprocessor = ColumnTransformer(
    transformers=[
        ('num', 'passthrough', numerical_features), # Keep numerical features as is
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features) # One-hot encode categorical features
    ],
    remainder='drop' # Drop any other columns not specified
)
# Split the data into training and testing sets
# Using a fixed random_state for reproducibility
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("Data preparation complete:")
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
Data preparation complete:
X_train shape: (828, 5)
X_test shape: (207, 5)
y_train shape: (828,)
y_test shape: (207,)
```

Model Training: Linear Regression

```python
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score
# Create a pipeline that first preprocesses and then applies Linear Regression
linear_reg_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])
# Train the Linear Regression model
linear_reg_model.fit(X_train, y_train)
# Make predictions on the test set
y_pred_lr = linear_reg_model.predict(X_test)
# Evaluate the Linear Regression model
mae_lr = mean_absolute_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)
print("Linear Regression Model Evaluation:")
print(f"Mean Absolute Error: {mae_lr:.2f}")
print(f"R-squared: {r2_lr:.2f}")
```

```
Linear Regression Model Evaluation:
Mean Absolute Error: 1.52
R-squared: 0.06
```

Model Training: Random Forest Regressor

Model Performance Comparison and Explanation

We trained two regression models to predict the `mass` of planets: Linear Regression and Random Forest Regressor. Let's compare their performance based on the Mean Absolute Error (MAE) and R-squared (R2) score.

Linear Regression Model Evaluation:

- **Mean Absolute Error (MAE):** 1.52
- **R-squared (R2):** 0.06

Random Forest Regressor Model Evaluation:

- **Mean Absolute Error (MAE):** 1.30
- **R-squared (R2):** 0.12

Analysis:

1. **Mean Absolute Error (MAE):**
   - The MAE for Linear Regression is 1.52, while for Random Forest Regressor, it is 1.30. A lower MAE indicates a better model, as it represents the average magnitude of the errors in a set of predictions, without considering their direction. In this case, the **Random Forest Regressor has a lower MAE**, suggesting that its predictions are, on average, closer to the actual `mass` values than those of the Linear Regression model.
2. **R-squared (R2) Score:**
   - The R-squared score for Linear Regression is 0.06, and for Random Forest Regressor, it is 0.12. R-squared represents the proportion of the variance in the dependent variable that is predictable from the independent variables. An R-squared of 1.0 indicates that the model explains all the variability of the response data around its mean, while an R-squared of 0 indicates that the model explains none of the variability of the response data around its mean.
   - Both models exhibit very low R-squared scores, which suggests that **neither model is able to explain a significant portion of the variance in the `mass` variable**. The **Random Forest Regressor's R-squared of 0.12 is slightly better** than the Linear Regression's 0.06, indicating it explains a tiny bit more of the variance, but still a very small amount.

Data Cleaning and Preprocessing: Handling Missing Values

Based on the previous analysis, the following columns have missing values:

- `mass` : 50.43% missing
- `distance` : 21.93% missing
- `orbital_period` : 4.15% missing

For numerical features, a common strategy to handle missing values is imputation. Given the nature of astronomical data, using the **median** is often a robust choice as it is less sensitive to outliers compared to the mean. We will apply median imputation to `orbital_period` , `mass` , and `distance` .

Note on 'mass' column: With over 50% missing values, imputing 'mass' with the median might introduce significant bias or dilute the statistical properties of the feature. Depending on the downstream task (e.g., model training), one might consider more advanced imputation techniques, or even dropping the column if its contribution after imputation is deemed insufficient or misleading. For this exercise, we will proceed with median imputation, but it's important to keep this limitation in mind for further analysis.

```python
# Impute missing values for numerical columns with their median
for column in ['orbital_period', 'mass', 'distance']:
    if planets_df[column].isnull().any(): # Check if the column actually has missing values
        median_value = planets_df[column].median()
        planets_df[column] = planets_df[column].fillna(median_value) # Removed inplace=True
        print(f"Missing values in '{column}' imputed with median: {median_value:.2f}")
print("\n--- Verifying Missing Values After Imputation ---")
# Recalculate and display missing values after imputation
missing_values_after = planets_df.isnull().sum()
missing_percentage_after = (planets_df.isnull().sum() / len(planets_df)) * 100
missing_info_after = pd.DataFrame({
    'Missing Count': missing_values_after,
    'Missing Percentage': missing_percentage_after
})
```

```python
})

# Filter to show only columns with missing values and sort by percentage
missing_info_after = missing_info_after[missing_info_after['Missing Count'] > 0].sort_values(by='Missing Percentage', ascending=False)

display(missing_info_after)

if missing_info_after.empty:
    print("All missing values have been successfully handled.")
else:
    print("Some missing values still remain.")
```

```
--- Verifying Missing Values After Imputation ---
  Missing Count  Missing Percentage
All missing values have been successfully handled.
```

```python
# Calculate the number of missing values for each column
missing_values = planets_df.isnull().sum()

# Calculate the percentage of missing values for each column
missing_percentage = (planets_df.isnull().sum() / len(planets_df)) * 100

# Create a DataFrame to display missing values
missing_info = pd.DataFrame({
    'Missing Count': missing_values,
    'Missing Percentage': missing_percentage
})

# Filter to show only columns with missing values and sort by percentage
missing_info = missing_info[missing_info['Missing Count'] > 0].sort_values(by='Missing Percentage', ascending=False)

print("\n--- Missing Values Information ---")
display(missing_info)

if missing_info.empty:
    print("No missing values found in the dataset.")
```

```
--- Missing Values Information ---
```

	Missing Count	Missing Percentage
mass	522	50.434783
distance	227	21.932367
orbital_period	43	4.154589

Task 4: Perform the following tasks on the given dataset (train the model)

- Load the MNIST dataset from PyTorch
- Display basic information and visualize sample input images with their class labels
- Perform data preprocessing
- Design an EfficientNet model for image classification
- Train the CNN model (10 epochs)
- Evaluate model performance using accuracy and loss metrics
- Visualize the training history (accuracy and loss curves)

```python
import torch
import torchvision
import torchvision.transforms as transforms

# Check if CUDA is available and set the device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')
```

```
Using device: cuda
```

Next, let's define the transformations to apply to the MNIST images. We'll convert them to tensors and normalize them.

```python
# Define transformations for the MNIST dataset
transform = transforms.Compose([
    transforms.ToTensor(), # Convert images to PyTorch tensors
    transforms.Normalize((0.1307,), (0.3081,)) # Normalize pixel values (mean and std for MNIST)
])
```

Now, let's load the MNIST training and test datasets. The `download=True` argument will download the dataset if it's not already present.

```python
# Load the MNIST training dataset
train_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=True,
    transform=transform,
    download=True
)

# Load the MNIST test dataset
test_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=False,
    transform=transform,
    download=True
)

print(f'Training dataset size: {len(train_dataset)} samples')
print(f'Test dataset size: {len(test_dataset)} samples')
```

```
Training dataset size: 60000 samples
Test dataset size: 10000 samples
```

Finally, we'll create data loaders for both the training and test sets. These will help us iterate over the data in batches during training and evaluation.

First, let's look at the basic information about the dataset, such as the number of classes and the class names.

```python
print(f'Number of classes in MNIST: {len(train_dataset.classes)}')
print(f'MNIST class names: {train_dataset.classes}')
```

```
Number of classes in MNIST: 10
MNIST class names: ['0 - zero', '1 - one', '2 - two', '3 - three', '4 - four', '5 - five', '6 - six', '7 - seven', '8 - eight', '9 - nine']
```

Now, let's visualize a few sample input images from the training dataset along with their class labels to get a better understanding of the data.

Model Training Setup

Before we start training, we need to define a **loss function** and an **optimizer**.

- **Loss Function**: For multi-class classification problems like MNIST, **Cross-Entropy Loss** is a standard choice. It measures the difference between the predicted probabilities and the true labels.
- **Optimizer**: The **Adam optimizer** is a popular and effective choice for deep learning models. It adapts the learning rate during training, often leading to faster convergence.

```python
# Define EfficientNet Model (moved from a previous cell to ensure 'model' is defined)
class EfficientNetClassifier(nn.Module):
    def __init__(self, num_classes=10):
        super(EfficientNetClassifier, self).__init__()
        # Load pre-trained EfficientNet-B0 model
        self.efficientnet = models.efficientnet_b0(pretrained=True)

        # Modify the classifier head
        # The original EfficientNet-B0 classifier has one linear layer
        # The in_features for the classifier is stored in _fc.in_features
        num_ftrs = self.efficientnet.classifier[1].in_features
        self.efficientnet.classifier[1] = nn.Linear(num_ftrs, num_classes)

        # Adapt the first convolutional layer for grayscale input (1 channel)
        # Original: Conv2d(3, ..., kernel_size=(3, 3)) for RGB
        # New: Conv2d(1, ..., kernel_size=(3, 3)) for grayscale
        original_conv1 = self.efficientnet.features[0][0]
        self.efficientnet.features[0][0] = nn.Conv2d(1, original_conv1.out_channels,
                                                     kernel_size=original_conv1.kernel_size,
                                                     stride=original_conv1.stride,
                                                     padding=original_conv1.padding,
                                                     bias=original_conv1.bias)

    def forward(self, x):
        return self.efficientnet(x)

# Instantiate the model
# Assuming 'train_dataset' and 'device' are defined in previous cells and are in scope.
model = EfficientNetClassifier(num_classes=len(train_dataset.classes))

# Move the model to the appropriate device (GPU if available, else CPU)
model = model.to(device)
```

```python
# Move the model to the appropriate device (GPU if available, else CPU)
model = model.to(device)

print(f'EfficientNet model created and moved to {device}. (Model definition moved here for error fix)')

# Define Loss Function and Optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001) # Learning rate can be tuned

print("Loss function (CrossEntropyLoss) and Optimizer (Adam) defined.")
```

```
EfficientNet model created and moved to cuda. (Model definition moved here for error fix)
Loss function (CrossEntropyLoss) and Optimizer (Adam) defined.
```

Training and Evaluation Loop

Now we'll define the training loop for the specified number of epochs. In each epoch, the model will iterate through the training data, make predictions, calculate the loss, and update its weights using the optimizer. After each epoch, we'll evaluate the model's performance on the test set to track its generalization ability.

```
batch_size = 64

train_loader = torch.utils.data.DataLoader(
    dataset=train_dataset,
    batch_size=batch_size,
    shuffle=True # Shuffle training data for better generalization
)

test_loader = torch.utils.data.DataLoader(
    dataset=test_dataset,
    batch_size=batch_size,
    shuffle=False # No need to shuffle test data
)

print(f'Number of training batches: {len(train_loader)}')
print(f'Number of test batches: {len(test_loader)}')

num_epochs = 10

train_losses = []
train_accuracies = []
test_losses = []
test_accuracies = []
```

```
print("Starting training...")

for epoch in range(num_epochs):
    model.train() # Set the model to training mode
    running_loss = 0.0
    correct_train = 0
    total_train = 0

    for i, (images, labels) in enumerate(train_loader):
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()

        outputs = model(images)
        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        running_loss += loss.item()

        _, predicted = torch.max(outputs.data, 1)
        total_train += labels.size(0)
        correct_train += (predicted == labels).sum().item()

    avg_train_loss = running_loss / len(train_loader)
    train_accuracy = 100 * correct_train / total_train
    train_losses.append(avg_train_loss)
    train_accuracies.append(train_accuracy)

    model.eval() # Set the model to evaluation mode
    test_loss = 0.0
```

```python
    test_loss = 0.0
    correct_test = 0
    total_test = 0
    with torch.no_grad(): # Disable gradient calculation for evaluation
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            test_loss += loss.item()

            _, predicted = torch.max(outputs.data, 1)
            total_test += labels.size(0)
            correct_test += (predicted == labels).sum().item()

    avg_test_loss = test_loss / len(test_loader)
    test_accuracy = 100 * correct_test / total_test
    test_losses.append(avg_test_loss)
    test_accuracies.append(test_accuracy)

    print(f'Epoch [{epoch+1}/{num_epochs}], ' \
          f'Train Loss: {avg_train_loss:.4f}, Train Acc: {train_accuracy:.2f}%, ' \
          f'Test Loss: {avg_test_loss:.4f}, Test Acc: {test_accuracy:.2f}%')

print("Training complete.")
```

```
Number of training batches: 938
Number of test batches: 157
Starting training...
Epoch [1/10], Train Loss: 0.2445, Train Acc: 92.61%, Test Loss: 0.0531, Test Acc: 98.27%
Epoch [2/10], Train Loss: 0.0814, Train Acc: 97.77%, Test Loss: 0.0486, Test Acc: 98.59%
Epoch [3/10], Train Loss: 0.0613, Train Acc: 98.33%, Test Loss: 0.0399, Test Acc: 98.81%
Epoch [4/10], Train Loss: 0.0464, Train Acc: 98.71%, Test Loss: 0.0360, Test Acc: 98.99%
Epoch [5/10], Train Loss: 0.0525, Train Acc: 98.57%, Test Loss: 0.0423, Test Acc: 98.79%
Epoch [6/10], Train Loss: 0.0458, Train Acc: 98.76%, Test Loss: 0.0396, Test Acc: 98.96%
Epoch [7/10], Train Loss: 0.0412, Train Acc: 98.87%, Test Loss: 0.0278, Test Acc: 99.08%
Epoch [8/10], Train Loss: 0.0400, Train Acc: 99.01%, Test Loss: 0.2397, Test Acc: 96.71%
Epoch [9/10], Train Loss: 0.0385, Train Acc: 99.02%, Test Loss: 0.0306, Test Acc: 98.97%
Epoch [10/10], Train Loss: 0.0320, Train Acc: 99.14%, Test Loss: 0.0307, Test Acc: 99.04%
Training complete.
```

Display Training Results

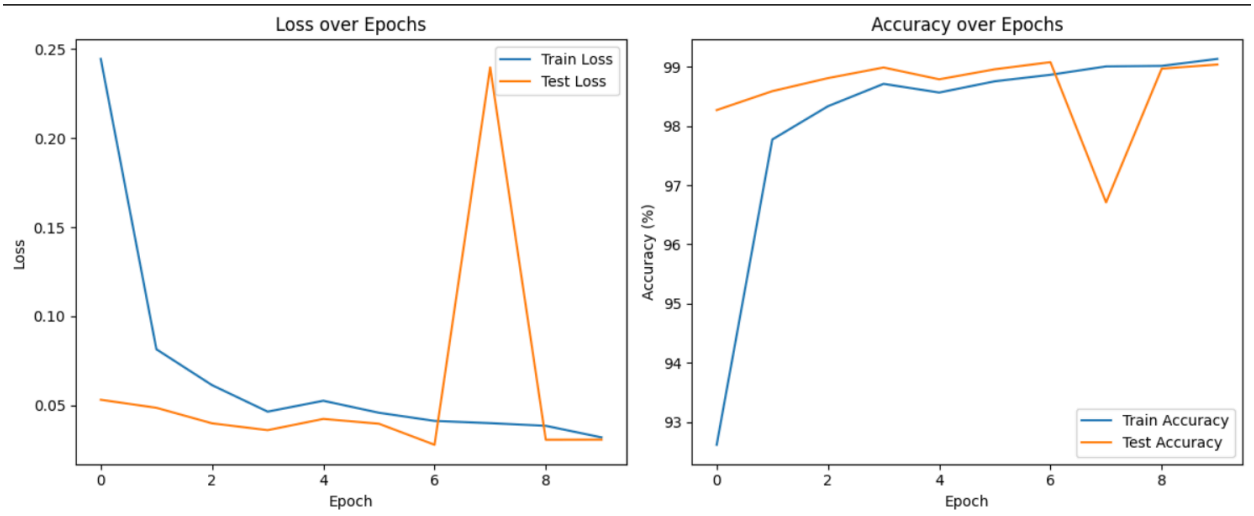
Let's visualize the training and test loss and accuracy curves over the epochs to understand the model's performance and identify potential overfitting.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(range(num_epochs), train_losses, label='Train Loss')
plt.plot(range(num_epochs), test_losses, label='Test Loss')
plt.title('Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(range(num_epochs), train_accuracies, label='Train Accuracy')
plt.plot(range(num_epochs), test_accuracies, label='Test Accuracy')
plt.title('Accuracy over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy (%)')
plt.legend()

plt.tight_layout()
plt.show()
```



EfficientNet is a family of image classification models that achieve state-of-the-art accuracy with significantly fewer parameters and FLOPs (floating-point operations) compared to other convolutional networks. They use a compound scaling method that uniformly scales all dimensions of depth, width, and resolution.

We will use `torchvision.models` to load a pre-trained EfficientNet-B0 model. Pre-trained models come with weights learned from a large dataset like ImageNet, which can be beneficial even when applying them to a different, but related, task like MNIST digit classification. We will then modify the classifier head to output 10 classes, matching the MNIST dataset.

```python
import torch.nn as nn
import torchvision.models as models

class EfficientNetClassifier(nn.Module):
    def __init__(self, num_classes=10):
        super(EfficientNetClassifier, self).__init__()
        # Load pre-trained EfficientNet-B0 model
        self.efficientnet = models.efficientnet_b0(pretrained=True)

        # Modify the classifier head
        # The original EfficientNet-B0 classifier has one linear layer
        # The in_features for the classifier is stored in _fc.in_features
        num_ftrs = self.efficientnet.classifier[1].in_features
        self.efficientnet.classifier[1] = nn.Linear(num_ftrs, num_classes)

        # Adapt the first convolutional layer for grayscale input (1 channel)
        # Original: Conv2d(3, ..., kernel_size=(3, 3)) for RGB
        # New: Conv2d(1, ..., kernel_size=(3, 3)) for grayscale
        original_conv1 = self.efficientnet.features[0][0]
        self.efficientnet.features[0][0] = nn.Conv2d(1, original_conv1.out_channels,
                                                     kernel_size=original_conv1.kernel_size,
                                                     stride=original_conv1.stride,
                                                     padding=original_conv1.padding,
                                                     bias=original_conv1.bias)
```

```python
    def forward(self, x):
        return self.efficientnet(x)

# Instantiate the model
model = EfficientNetClassifier(num_classes=len(train_dataset.classes))

# Move the model to the appropriate device (GPU if available, else CPU)
model = model.to(device)

print(f'EfficientNet model created and moved to {device}.')
print(f'Number of parameters: {sum(p.numel() for p in model.parameters() if p.requires_grad)}')
```

```
EfficientNet model created and moved to cuda.
Number of parameters: 4019782
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, p
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 an
  warnings.warn(msg)
```

Data Augmentation

Data augmentation is a technique used to increase the diversity of training data by applying random transformations, such as rotations or shifts. This helps to prevent overfitting and improve the model's ability to generalize to new, unseen data.

We will define a new set of transformations specifically for the training data (`train_transform`) that includes augmentation, while the test data (`test_transform`) will only have the necessary conversion to tensor and normalization.

```python
# Define transformations for the training dataset with augmentation
train_transform = transforms.Compose([
    transforms.RandomRotation(degrees=15), # Randomly rotate images by up to 15 degrees
    transforms.RandomAffine(degrees=0, translate=(0.1, 0.1)), # Randomly translate images by up to 10% horizontally/vertically
    transforms.ToTensor(), # Convert images to PyTorch tensors
    transforms.Normalize((0.1307,), (0.3081,)) # Normalize pixel values (mean and std for MNIST)
])

# Define transformations for the test dataset (no augmentation)
test_transform = transforms.Compose([
    transforms.ToTensor(), # Convert images to PyTorch tensors
    transforms.Normalize((0.1307,), (0.3081,)) # Normalize pixel values (mean and std for MNIST)
])

print("New training and test transforms defined with data augmentation for training.")
```

```
New training and test transforms defined with data augmentation for training.
```

```python
# Re-load the MNIST training dataset with the new train_transform
train_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=True,
    transform=train_transform,
    download=False # Data should already be downloaded
)

# Re-load the MNIST test dataset with the new test_transform
test_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=False,
    transform=test_transform,
    download=False # Data should already be downloaded
)

print(f'Training dataset size: {len(train_dataset)} samples')
print(f'Test dataset size: {len(test_dataset)} samples')
print("Datasets re-initialized with updated transforms.")
```

```
Training dataset size: 60000 samples
Test dataset size: 10000 samples
Datasets re-initialized with updated transforms.
```

```python
# Re-create data loaders with the re-initialized datasets
train_loader = torch.utils.data.DataLoader(
    dataset=train_dataset,
    batch_size=batch_size,
    shuffle=True # Shuffle training data for better generalization
)

test_loader = torch.utils.data.DataLoader(
```

```python
test_loader = torch.utils.data.DataLoader(
    dataset=test_dataset,
    batch_size=batch_size,
    shuffle=False # No need to shuffle test data
)

print(f'Number of training batches: {len(train_loader)}')
print(f'Number of test batches: {len(test_loader)}')
print("Data loaders re-created.")
```

```
Number of training batches: 938
Number of test batches: 157
Data loaders re-created.
```

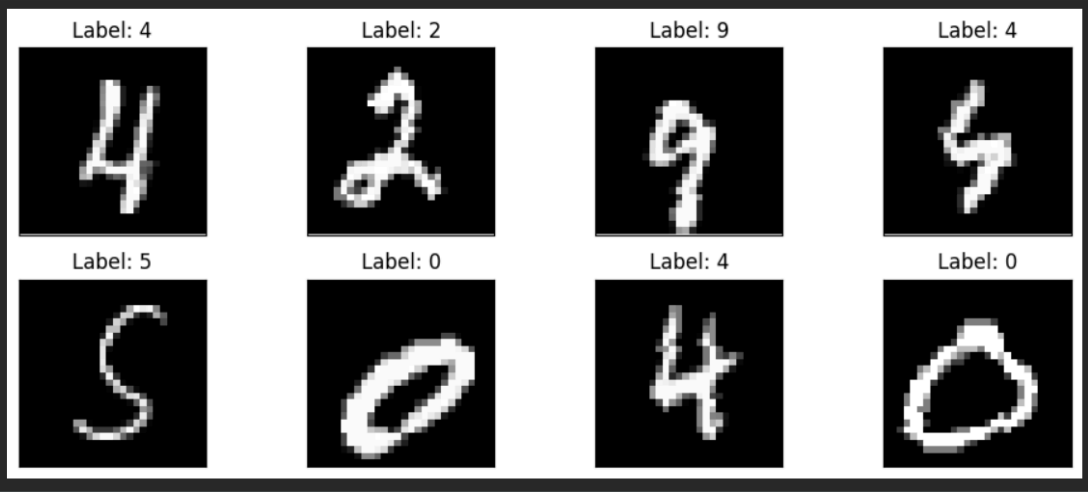
```python
import matplotlib.pyplot as plt
import numpy as np

# Get a batch of training data
images, labels = next(iter(train_loader))

# Denormalize images for display (assuming mean 0.1307 and std 0.3081 for MNIST)
def denormalize(image_tensor):
    mean = 0.1307
    std = 0.3081
    return image_tensor * std + mean

# Create a figure and a set of subplots
fig = plt.figure(figsize=(10, 4))
for i in range(8): # Display first 8 images in the batch
    ax = fig.add_subplot(2, 4, i + 1, xticks=[], yticks=[])
    # Convert tensor to numpy array for display, remove channel dimension, and denormalize
    img = denormalize(images[i]).squeeze().numpy()
    ax.imshow(img, cmap='gray')
    ax.set_title(f'Label: {labels[i].item()}')
plt.tight_layout()
```

```python
# Create a figure and a set of subplots
fig = plt.figure(figsize=(10, 4))
for i in range(8): # Display first 8 images in the batch
    ax = fig.add_subplot(2, 4, i + 1, xticks=[], yticks=[])
    # Convert tensor to numpy array for display, remove channel dimension, and denormalize
    img = denormalize(images[i]).squeeze().numpy()
    ax.imshow(img, cmap='gray')
    ax.set_title(f'Label: {labels[i].item()}')
plt.tight_layout()
plt.show()
```




```python
# Define batch size
batch_size = 64

# Create data loaders
train_loader = torch.utils.data.DataLoader(
    dataset=train_dataset,
    batch_size=batch_size,
    shuffle=True # Shuffle training data for better generalization
)

test_loader = torch.utils.data.DataLoader(
    dataset=test_dataset,
    batch_size=batch_size,
    shuffle=False # No need to shuffle test data
)

print(f'Number of training batches: {len(train_loader)}')
print(f'Number of test batches: {len(test_loader)}')

# Example of getting one batch of data
images, labels = next(iter(train_loader))
print(f'Shape of one batch of images: {images.shape}')
print(f'Shape of one batch of labels: {labels.shape}')
```

```
Number of training batches: 938
Number of test batches: 157
Shape of one batch of images: torch.Size([64, 1, 28, 28])
Shape of one batch of labels: torch.Size([64])
```

Task 7: Healthcare Data Preparation

Task: Clean and preprocess a healthcare dataset using AI scripts.

Instructions:

- Handle missing values in `blood_pressure`, `cholesterol`
- Encode categorical features
- Scale numerical features (min-max or standard scaling)
- Split dataset into training and testing sets

Dataset link: https://www.kaggle.com/datasets/abdallaahmed77/healthcare-risk-factors-dataset

Expected Output:

- A preprocessed dataset ready for machine learning models

Download Dataset from Kaggle

To download the dataset directly from Kaggle, you'll need a `kaggle.json` file containing your API credentials.

1. **Generate Kaggle API Key**: Go to your Kaggle account page (https://www.kaggle.com/your-username/account) and click on 'Create New API Token'. This will download a `kaggle.json` file.
2. **Upload `kaggle.json` to Colab**: Click on the folder icon on the left sidebar (Files), then click on 'Upload to session storage' icon. Upload your `kaggle.json` file.
3. **Move `kaggle.json`**: Run the following code to move `kaggle.json` to the correct directory (`~/.kaggle/`).

```
# Install Kaggle API client
!pip install kaggle --quiet

# Make a directory for Kaggle and move the JSON file
!mkdir -p ~/.kaggle
!mv kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json

print("Kaggle API key setup complete.")
```

```
mv: cannot stat 'kaggle.json': No such file or directory
chmod: cannot access '/root/.kaggle/kaggle.json': No such file or directory
Kaggle API key setup complete.
```

Now, let's download the dataset. The dataset is named `healthcare-risk-factors-dataset` and contains a `healthcare_dataset.csv` file.

```python
# Download the dataset
!kaggle datasets download -d abdallaahmed77/healthcare-risk-factors-dataset

# Unzip the downloaded file
import zipfile
import os

zip_file_name = 'healthcare-risk-factors-dataset.zip'
if os.path.exists(zip_file_name):
    with zipfile.ZipFile(zip_file_name, 'r') as zip_ref:
        zip_ref.extractall('.')
    print(f"'{zip_file_name}' extracted.")
else:
    print(f"Error: '{zip_file_name}' not found. Please ensure it downloaded correctly.")

# List files to verify
!ls
```

```
Dataset URL: https://www.kaggle.com/datasets/abdallaahmed77/healthcare-risk-factors-dataset
License(s): CC0-1.0
Downloading healthcare-risk-factors-dataset.zip to /content
100% 1.28M/1.28M [00:00<00:00, 2.44MB/s]

'healthcare-risk-factors-dataset.zip' extracted.
dirty_v3_path.csv  healthcare-risk-factors-dataset.zip  sample_data
```

With the dataset downloaded and extracted, the next step is to load it into a pandas DataFrame and display the first few rows to understand its structure.

```python
import pandas as pd

# Load the dataset into a pandas DataFrame
df = pd.read_csv('dirty_v3_path.csv')

# Display the first 5 rows of the DataFrame
print("First 5 rows of the dataset:")
display(df.head())
```

```
First 5 rows of the dataset:
```

	Age	Gender	Medical Condition	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	HbA1c	Smoking	Alcohol	Physical Activity	Diet Score	Family History	Stress Level	Sleep Hours	ra
0	46.0	Male	Diabetes	137.04	135.27	28.90	96.04	6	231.88	210.56	7.61	0	0	-0.20	3.54	0	5.07	6.05	
1	22.0	Male	Healthy	71.58	113.27	26.29	97.54	2	165.57	129.41	4.91	0	0	8.12	5.90	0	5.87	7.72	
2	50.0	NaN	Asthma	95.24	NaN	22.53	90.31	2	214.94	165.35	5.60	0	0	5.01	4.65	1	3.09	4.82	
3	57.0	NaN	Obesity	NaN	130.53	38.47	96.60	5	197.71	182.13	6.92	0	0	3.16	3.37	0	3.01	5.33	
4	66.0	Female	Hypertension	95.15	178.17	31.12	94.90	4	259.53	115.85	5.98	0	1	3.56	3.40	0	6.38	6.64	

Let's also look at the general information about the dataset, including data types and non-null counts, to identify missing values and categorical features.

```python
print("\nDataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 20 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Age                25500 non-null  float64
 1   Gender             25500 non-null  object
 2   Medical Condition  25500 non-null  object
 3   Glucose            25500 non-null  float64
 4   Blood Pressure     25500 non-null  float64
 5   BMI                30000 non-null  float64
 6   Oxygen Saturation  30000 non-null  float64
 7   LengthOfStay       30000 non-null  int64
 8   Cholesterol        30000 non-null  float64
 9   Triglycerides      30000 non-null  float64
 10  HbA1c              30000 non-null  float64
 11  Smoking            30000 non-null  int64
 12  Alcohol            30000 non-null  int64
 13  Physical Activity  30000 non-null  float64
 14  Diet Score         30000 non-null  float64
 15  Family History     30000 non-null  int64
 16  Stress Level       30000 non-null  float64
 17  Sleep Hours        30000 non-null  float64
 18  random_notes       30000 non-null  object
 19  noise_col          30000 non-null  float64
dtypes: float64(13), int64(4), object(3)
memory usage: 4.6+ MB
```

Splitting the Dataset into Training and Testing Sets

Finally, I'll split the preprocessed dataset into features (X) and the target variable (y). For this dataset, `LengthOfStay` will be considered the target variable. Then, I'll use `train_test_split` from `sklearn.model_selection` to create training and testing sets, typically with an 80/20 split.

```python
from sklearn.model_selection import train_test_split

# Define features (X) and target (y)
X = df.drop(['LengthOfStay', 'random_notes', 'noise_col'], axis=1)
y = df['LengthOfStay']

# Split the dataset into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Dataset split into training and testing sets:")
print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")

print("\nFirst 5 rows of X_train:")
display(X_train.head())
```

```
Dataset split into training and testing sets:
X_train shape: (24000, 22)
X_test shape: (6000, 22)
y_train shape: (24000,)
y_test shape: (6000,)

First 5 rows of X_train:
```

```
First 5 rows of X_train:
```

	Age	Glucose	Blood Pressure	BMI	Oxygen Saturation	Cholesterol	Triglycerides	HbA1c	Smoking	Alcohol	...	Family History	Stress Level	Sleep Hours	Gender_Male	Medical Condition_Asthma
21753	0.443038	0.286428	0.531747	0.314762	0.684680	0.519342	0.551566	0.346916	0	0	...	1	0.498044	0.302511	False	False
251	0.645570	0.670378	0.532930	0.258032	0.696194	0.452406	0.471407	0.511013	0	0	...	1	0.376188	0.226027	False	False
22941	0.253165	0.243838	0.421191	0.545547	0.700423	0.408506	0.347620	0.240088	0	0	...	0	0.479039	0.646119	True	False
618	0.848101	0.274422	0.421191	0.286905	0.679041	0.371573	0.424424	0.270925	1	0	...	0	0.446618	0.361872	False	False
17090	0.911392	0.248365	0.515775	0.448556	0.590226	0.500000	0.675038	0.350220	0	1	...	0	0.502515	0.625571	True	False

5 rows × 22 columns

```python
from sklearn.preprocessing import MinMaxScaler

# Identify numerical columns to scale
# Exclude binary features, encoded categorical features, and irrelevant columns
numerical_cols = df.select_dtypes(include=['float64', 'int64']).columns.tolist()

# Remove columns that are already binary or encoded (from previous steps)
exclude_cols = ['LengthOfStay', 'Smoking', 'Alcohol', 'Family History', 'noise_col']

# Also exclude the one-hot encoded columns which are already boolean
# The names for these columns start with 'Gender_' and 'Medical Condition_'
encoded_cols = [col for col in df.columns if col.startswith('Gender_') or col.startswith('Medical Condition_')]

# The 'random_notes' column is an object type and should also be excluded from numerical scaling
exclude_cols.append('random_notes')

# Filter out excluded columns from numerical_cols
numerical_cols = [col for col in numerical_cols if col not in exclude_cols and col not in encoded_cols]

# Apply Min-Max Scaling
scaler = MinMaxScaler()
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])

print("DataFrame after Min-Max Scaling (first 5 rows):")
display(df.head())
print("\nUpdated DataFrame Info after scaling:")
df.info()
```

```
DataFrame after Min-Max Scaling (first 5 rows):
```

	Age	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	HbA1c	Smoking	...	Sleep Hours	random_notes	noise_col	Gender_Male	Conditio
0	0.455696	0.391428	0.401144	0.431680	0.670348	6	0.518390	0.524877	0.476872	0	...	0.509132	lorem	-137.057211	True	
1	0.151899	0.171904	0.256540	0.378609	0.705592	2	0.265915	0.342102	0.179515	0	...	0.699772	ipsum	-11.230610	True	
2	0.506329	0.251249	0.421191	0.302155	0.535714	2	0.453891	0.423050	0.255507	0	...	0.368721	ipsum	98.331195	False	
3	0.594937	0.302425	0.369988	0.626271	0.683506	5	0.388288	0.460844	0.400881	0	...	0.426941	lorem	44.187175	False	
4	0.708861	0.250947	0.683121	0.476820	0.643562	4	0.623667	0.311561	0.297357	0	...	0.576484	lorem	44.831426	False	

5 rows × 25 columns

```
Updated DataFrame Info after scaling:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 25 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Age                     30000 non-null  float64
 1   Glucose                 30000 non-null  float64
 2   Blood Pressure          30000 non-null  float64
 3   BMI                     30000 non-null  float64
 4   Oxygen Saturation       30000 non-null  float64
 5   LengthOfStay            30000 non-null  int64
 6   Cholesterol             30000 non-null  float64
 7   Triglycerides           30000 non-null  float64
 8   HbA1c                   30000 non-null  float64
 9   Smoking                 30000 non-null  int64
 10  Alcohol                 30000 non-null  int64
 11  Physical Activity       30000 non-null  float64
 12  Diet Score              30000 non-null  float64
```

Encoding Categorical Features

Next, I'll identify categorical columns and apply one-hot encoding to them. This will convert categorical variables into a format that can be provided to machine learning algorithms. The columns `Gender` and `Medical Condition` are identified as categorical features that require encoding.

```python
# Identify categorical columns to encode (excluding 'random_notes' and 'noise_col' which seem irrelevant for modeling)
categorical_cols = ['Gender', 'Medical Condition']

# Apply one-hot encoding
df = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

print("DataFrame after one-hot encoding:")
display(df.head())
print("\nUpdated DataFrame Info after encoding:")
df.info()
```

```
DataFrame after one-hot encoding:
```

	Age	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	HbA1c	Smoking	...	Sleep Hours	random_notes	noise_col	Gender_Male	Medical Condition_Asthma	Medical Condition_Cancer	Conditi
0	46.0	137.04	135.27	28.90	96.04	6	231.88	210.56	7.61	0	...	6.05	lorem	-137.057211	True	False	False	
1	22.0	71.58	113.27	26.29	97.54	2	165.57	129.41	4.91	0	...	7.72	ipsum	-11.230610	True	False	False	
2	50.0	95.24	138.32	22.53	90.31	2	214.94	165.35	5.60	0	...	4.82	ipsum	98.331195	False	True	False	
3	57.0	110.50	130.53	38.47	96.60	5	197.71	182.13	6.92	0	...	5.33	lorem	44.187175	False	False	False	
4	66.0	95.15	178.17	31.12	94.90	4	259.53	115.85	5.98	0	...	6.64	lorem	44.831426	False	False	False	

5 rows × 25 columns

```
Updated DataFrame Info after encoding:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 25 columns):
 #   Column                        Non-Null Count  Dtype
```

```
Updated DataFrame Info after encoding:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 25 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   Age                           30000 non-null  float64
 1   Glucose                       30000 non-null  float64
 2   Blood Pressure                30000 non-null  float64
 3   BMI                           30000 non-null  float64
 4   Oxygen Saturation             30000 non-null  float64
 5   LengthOfStay                  30000 non-null  int64
 6   Cholesterol                   30000 non-null  float64
 7   Triglycerides                 30000 non-null  float64
 8   HbA1c                         30000 non-null  float64
 9   Smoking                       30000 non-null  int64
 10  Alcohol                       30000 non-null  int64
 11  Physical Activity             30000 non-null  float64
 12  Diet Score                    30000 non-null  float64
 13  Family History                30000 non-null  int64
 14  Stress Level                  30000 non-null  float64
 15  Sleep Hours                   30000 non-null  float64
 16  random_notes                  30000 non-null  object
 17  noise_col                     30000 non-null  float64
 18  Gender_Male                   30000 non-null  bool
 19  Medical Condition_Asthma      30000 non-null  bool
 20  Medical Condition_Cancer      30000 non-null  bool
 21  Medical Condition_Diabetes    30000 non-null  bool
 22  Medical Condition_Healthy     30000 non-null  bool
 23  Medical Condition_Hypertension 30000 non-null  bool
 24  Medical Condition_Obesity     30000 non-null  bool
dtypes: bool(7), float64(13), int64(4), object(1)
memory usage: 4.3+ MB
```

Handling Missing Values

First, let's check the percentage of missing values for each column to understand the extent of the problem. Then, I'll impute missing values in `Blood Pressure`, `Age`, `Glucose`, `Gender`, and `Medical Condition`.

```python
print("\nMissing values percentage per column:")
print(df.isnull().sum() / len(df) * 100)

# Impute missing values in numerical columns with the median
for col in ['Age', 'Glucose', 'Blood Pressure']:
    if df[col].isnull().any():
        median_val = df[col].median()
        df[col].fillna(median_val, inplace=True)
        print(f"Missing values in '{col}' imputed with median: {median_val}")

# Impute missing values in categorical columns with the mode
for col in ['Gender', 'Medical Condition']:
    if df[col].isnull().any():
        mode_val = df[col].mode()[0]
        df[col].fillna(mode_val, inplace=True)
        print(f"Missing values in '{col}' imputed with mode: {mode_val}")

print("\nMissing values after imputation:")
print(df.isnull().sum())

# Verify the info again after imputation
print("\nDataset Information after handling missing values:")
df.info()
```

```
Missing values percentage per column:
Age                 15.0
Gender              15.0
Medical Condition   15.0
Glucose             15.0
Blood Pressure      15.0
BMI                  0.0
Oxygen Saturation    0.0
```

```
Alcohol               0.0
Physical Activity     0.0
Diet Score            0.0
Family History        0.0
Stress Level          0.0
Sleep Hours           0.0
random_notes          0.0
noise_col             0.0
dtype: float64
Missing values in 'Age' imputed with median: 55.0
Missing values in 'Glucose' imputed with median: 110.5
Missing values in 'Blood Pressure' imputed with median: 138.32
Missing values in 'Gender' imputed with mode: Female
Missing values in 'Medical Condition' imputed with mode: Hypertension

Missing values after imputation:
Age                  0
Gender               0
Medical Condition    0
Glucose              0
Blood Pressure       0
BMI                  0
Oxygen Saturation    0
LengthOfStay         0
Cholesterol          0
Triglycerides        0
HbA1c                0
Smoking              0
Alcohol              0
Physical Activity    0
Diet Score           0
Family History       0
Stress Level         0
Sleep Hours          0
random_notes         0
noise_col            0
dtype: int64
```

```
Dataset Information after handling missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 20 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Age                 30000 non-null  float64
 1   Gender              30000 non-null  object
 2   Medical Condition   30000 non-null  object
 3   Glucose             30000 non-null  float64
 4   Blood Pressure      30000 non-null  float64
 5   BMI                 30000 non-null  float64
 6   Oxygen Saturation   30000 non-null  float64
 7   LengthOfStay        30000 non-null  int64
 8   Cholesterol         30000 non-null  float64
 9   Triglycerides       30000 non-null  float64
 10  HbA1c               30000 non-null  float64
 11  Smoking             30000 non-null  int64
 12  Alcohol             30000 non-null  int64
 13  Physical Activity   30000 non-null  float64
 14  Diet Score          30000 non-null  float64
 15  Family History      30000 non-null  int64
 16  Stress Level        30000 non-null  float64
 17  Sleep Hours         30000 non-null  float64
 18  random_notes        30000 non-null  object
 19  noise_col           30000 non-null  float64
dtypes: float64(13), int64(4), object(3)
memory usage: 4.6+ MB
/tmp/ipykernel_9914/2090148516.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on th


  df[col].fillna(median_val, inplace=True)
/tmp/ipykernel_9914/2090148516.py:15: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on th
```